

- Description

This module manages a smart wind turbine power regulation system, ensuring that:

1. The system continuously monitors the current wind speed (in m/s).
2. If the wind speed falls below a predefined minimum threshold, power generation is halted due to insufficient wind force.
3. If the wind speed exceeds a predefined maximum threshold, the turbine is shut down to prevent damage.
4. If the wind speed is within the optimal range, the turbine operates normally, adjusting power output based on a numerical formula.

This system helps maintain optimal power generation efficiency while ensuring turbine safety.

- Specification

module SmartWindTurbineControl

var currentWindSpeed, minWindSpeed, maxWindSpeed, maxPowerOutput, currentPowerOutput: real

process updateWindSpeed(newWindSpeed: real) returns (currentWindSpeed: real, currentPowerOutput: real)

functional scenario form:

currentWindSpeed < minWindSpeed && currentPowerOutput = 0.0, output "ALERT: Wind speed too low (" + currentWindSpeed + " m/s). Power generation halted.", Power generation halted.

||

currentWindSpeed > maxWindSpeed && currentPowerOutput = 0.0, output "ALERT: Wind speed too high (" + currentWindSpeed + " m/s). Turbine shut down for safety.", Turbine shut down for safety.

||

currentWindSpeed >= minWindSpeed && currentPowerOutput = maxPowerOutput * (currentWindSpeed / maxWindSpeed)^3, output "Wind speed: " + currentWindSpeed + " m/s. Power output: " + currentPowerOutput + " MW. Turbine operating normally.", Power output adjusted based on wind speed.

end process

end module

- Java Code

```
package code;
public class c94_SmartWindTurbine {
    private double minWindSpeed;
    private double maxWindSpeed;
    private double currentWindSpeed;
    private double maxPowerOutput;
    private double currentPowerOutput;
```

```

    public c94_SmartWindTurbine(double minWindSpeed, double maxWindSpeed,
double maxPowerOutput, double initialWindSpeed) {
        this.minWindSpeed = minWindSpeed;
        this.maxWindSpeed = maxWindSpeed;
        this.maxPowerOutput = maxPowerOutput;
        this.currentWindSpeed = initialWindSpeed;
        this.currentPowerOutput = 0.0;
    }
    public void updateWindSpeed(double newWindSpeed) {
        currentWindSpeed = newWindSpeed;
        adjustPowerOutput();
        System.out.printf("Current Wind Speed: %.2f m/s, Power Output:
%.2f MW%n",
            currentWindSpeed, currentPowerOutput);
        if (currentWindSpeed < minWindSpeed) {
            System.out.println("ALERT: Wind speed too low. Power genera
tion halted.");
        } else if (currentWindSpeed > maxWindSpeed) {
            System.out.println("ALERT: Wind speed too high. Turbine shu
t down for safety.");
        } else {
            System.out.println("Turbine operating normally.");
        }
    }
    private void adjustPowerOutput() {
        if (currentWindSpeed < minWindSpeed || currentWindSpeed > maxWi
ndSpeed) {
            currentPowerOutput = 0.0;
        } else {
            currentPowerOutput = maxPowerOutput * Math.pow((currentWind
Speed / maxWindSpeed), 3);
        }
    }
    public double getCurrentWindSpeed() {
        return currentWindSpeed;
    }
    public double getCurrentPowerOutput() {
        return currentPowerOutput;
    }
    public static void main(String[] args) {
        c94_SmartWindTurbine turbine = new c94_SmartWindTurbine(3.0, 25.0,
2.0, 10.0);
        // testcase-VT:
        // turbine.updateWindSpeed(2.0);

```

```
//      turbine.updateWindSpeed(12.0);
//      turbine.updateWindSpeed(28.0);

//      // testcase-FT:
//      turbine.updateWindSpeed(1.0574665499190092E307);
//      turbine.updateWindSpeed(-1.7976931348623157E308);
//      turbine.updateWindSpeed(1.5205429148608833E298);

//      // testcase-Z3:
//      turbine.updateWindSpeed(2.0);
//      turbine.updateWindSpeed(12.0);
//      turbine.updateWindSpeed(28.0);

//      // testcase-UVT:
//      turbine.updateWindSpeed(2.0);
//      turbine.updateWindSpeed(12.0);
//      turbine.updateWindSpeed(28.0);
//  }
}
```

- Coverage report

1. Test case number: 3
2. V-Method testing branch coverage: 100%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
▼ VibrationTestingTool	1.0 %	8	834	842
▼ src	1.0 %	8	834	842
▼ code	1.0 %	8	834	842
▼ c94_SmartWindTurbine.java	100.0 %	8	0	8
▼ c94_SmartWindTurbine	100.0 %	8	0	8
main(String[])		0	0	0
c94_SmartWindTurbine(double,		0	0	0
adjustPowerOutput()	100.0 %	4	0	4
updateWindSpeed(double)	100.0 %	4	0	4

3. Vibration testing instruction coverage: 94.9%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
▼ VibrationTestingTool	0.5 %	112	21,587	21,699
▼ src	0.5 %	112	21,587	21,699
▼ code	0.5 %	112	21,587	21,699
▼ c94_SmartWindTurbine.java	94.9 %	112	6	118
▼ c94_SmartWindTurbine	94.9 %	112	6	118
main(String[])	100.0 %	18	0	18
c94_SmartWindTurbine(double,	100.0 %	18	0	18
adjustPowerOutput()	100.0 %	29	0	29
updateWindSpeed(double)	100.0 %	47	0	47

4. Fuzz testing branch coverage: 75%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
▼ VibrationTestingTool	0.7 %	6	836	842
▼ src	0.7 %	6	836	842
▼ code	0.7 %	6	836	842
▼ c94_SmartWindTurbine.java	75.0 %	6	2	8
▼ c94_SmartWindTurbine	75.0 %	6	2	8
main(String[])		0	0	0
c94_SmartWindTurbine(double,		0	0	0
adjustPowerOutput()	75.0 %	3	1	4
updateWindSpeed(double)	75.0 %	3	1	4

5. Fuzz testing instruction coverage: 82.2%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
▼ VibrationTestingTool	0.4 %	97	21,602	21,699
▼ src	0.4 %	97	21,602	21,699
▼ code	0.4 %	97	21,602	21,699
▼ c94_SmartWindTurbine.java	82.2 %	97	21	118
▼ c94_SmartWindTurbine	82.2 %	97	21	118
main(String[])	100.0 %	18	0	18
c94_SmartWindTurbine(double,	100.0 %	18	0	18
adjustPowerOutput()	58.6 %	17	12	29
updateWindSpeed(double)	93.6 %	44	3	47

6. Z3 branch coverage: 100%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
▼ VibrationTestingTool	1.0 %	8	834	842
▼ src	1.0 %	8	834	842
▼ code	1.0 %	8	834	842
▼ c94_SmartWindTurbine.java	100.0 %	8	0	8
▼ c94_SmartWindTurbine	100.0 %	8	0	8
main(String[])		0	0	0
c94_SmartWindTurbine(double,		0	0	0
adjustPowerOutput()	100.0 %	4	0	4
updateWindSpeed(double)	100.0 %	4	0	4

7. Z3 instruction coverage: 94.9%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
▼ VibrationTestingTool	0.5 %	112	21,587	21,699
▼ src	0.5 %	112	21,587	21,699
▼ code	0.5 %	112	21,587	21,699
▼ c94_SmartWindTurbine.java	94.9 %	112	6	118
▼ c94_SmartWindTurbine	94.9 %	112	6	118
main(String[])	100.0 %	18	0	18
c94_SmartWindTurbine(double,	100.0 %	18	0	18
adjustPowerOutput()	100.0 %	29	0	29
updateWindSpeed(double)	100.0 %	47	0	47

8. Unimproved V-method branch coverage: 100%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
▼ VibrationTestingTool	1.0 %	8	834	842
▼ src	1.0 %	8	834	842
▼ code	1.0 %	8	834	842
▼ c94_SmartWindTurbine.java	100.0 %	8	0	8
▼ c94_SmartWindTurbine	100.0 %	8	0	8
main(String[])		0	0	0
c94_SmartWindTurbine(double,		0	0	0
adjustPowerOutput()	100.0 %	4	0	4
updateWindSpeed(double)	100.0 %	4	0	4

9. Unimproved V-method instruction coverage: 94.9%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
▼ VibrationTestingTool	0.5 %	112	21,587	21,699
▼ src	0.5 %	112	21,587	21,699
▼ code	0.5 %	112	21,587	21,699
▼ c94_SmartWindTurbine.java	94.9 %	112	6	118
▼ c94_SmartWindTurbine	94.9 %	112	6	118
main(String[])	100.0 %	18	0	18
c94_SmartWindTurbine(double,	100.0 %	18	0	18
adjustPowerOutput()	100.0 %	29	0	29
updateWindSpeed(double)	100.0 %	47	0	47